L.E.O

Linguistic Executive Officer

Project Report

An Autonomous AI Assistant with Real-Time Voice,
Agentic Task Planning, and System Control

Author: Varun Nayaka

Date: April 2026

GitHub: github.com/VARUN-NAYAKA/LEO

Table of Contents

1. Abstract
2. Introduction
3. Summary
4. Problem Statement
5. Objectives of the Project
6. System Architecture
7. Proposed Methodology
8. Tools Used
9. Design Diagrams
10. Progress of Work
11. Modules Planned and Completed
12. References

Chapter 1: Abstract

LEO (Linguistic Executive Officer) is a fully autonomous, voice-controlled AI assistant engineered to transform the way users interact with their operating systems. Built on Google's Gemini 2.5 Flash Live API, LEO provides ultra-low-latency real-time voice conversation with native audio processing capabilities. The system features a modular architecture comprising 16 specialized action modules that enable comprehensive computer control, including browser automation via Playwright, file management, application launching, screen and webcam analysis, code generation, and multi-step agentic task execution.

LEO incorporates a persistent memory system that maintains user context across sessions, storing personal information, preferences, and project details in a structured JSON format. The agentic planning subsystem decomposes complex user goals into executable step sequences, with built-in error recovery that can retry, skip, replan, or abort failed operations. The system operates entirely locally on Windows, macOS, and Linux, requiring only a free Gemini API key, and features a custom Tkinter-based HUD interface with real-time animations, status indicators, and audio waveform visualization.

Chapter 2: Introduction

The evolution of artificial intelligence has progressed from simple rule-based chatbots to sophisticated conversational agents. However, most AI assistants remain confined to text-based interactions and lack the ability to directly manipulate the user's computing environment. LEO bridges this gap by combining real-time voice interaction with autonomous system control capabilities.

LEO stands for Linguistic Executive Officer, reflecting its role as an intelligent executive that manages and controls computing tasks through natural language. Unlike conventional virtual assistants such as Siri, Alexa, or Google Assistant, LEO operates directly on the user's machine with full system access, enabling it to open applications, control browsers, manage files, write code, and execute complex multi-step workflows autonomously.

The project leverages Google's Gemini 2.5 Flash model with native audio capabilities for real-time bidirectional voice communication. The Live API enables streaming audio input and output with function calling support, allowing LEO to listen, understand, act, and respond in a continuous conversational loop. This architecture eliminates the traditional speech-to-text and text-to-speech pipeline, resulting in significantly lower latency and more natural interactions.

Chapter 3: Summary

LEO is a cross-platform AI assistant that transforms any computer into an intelligent, voice-controlled workstation. The system is built around five core pillars:

- Real-Time Voice Engine: Bidirectional audio streaming at 16kHz input / 24kHz output via Gemini Live API with native audio processing.
- System Control Suite: 16 specialized action modules providing comprehensive control over applications, browsers, files, system settings, and more.
- Agentic Task Planning: An autonomous planning-execution pipeline that breaks complex goals into tool-call sequences with intelligent error recovery.
- Visual Intelligence: Real-time screen capture and webcam analysis using Gemini's multimodal vision capabilities.
- Persistent Memory: A structured long-term memory system that stores user identity, preferences, projects, and relationships across sessions.

The project consists of approximately 11,000 lines of Python code organized into a modular architecture with clear separation of concerns. The UI features a custom-built animated HUD with rotating rings, pulse effects, and real-time audio waveform visualization, providing an immersive Iron Man JARVIS-inspired interface.

Chapter 4: Problem Statement

Current AI assistants suffer from several critical limitations that prevent them from being truly useful as personal computing companions:

- Lack of System Control: Existing assistants like ChatGPT, Siri, and Alexa cannot directly interact with the user's operating system, applications, or file system.
- No Persistent Context: Most AI systems treat every conversation as independent, forgetting user preferences, ongoing projects, and personal context between sessions.
- Limited Autonomy: Current solutions require step-by-step user guidance and cannot autonomously plan and execute multi-step workflows.
- High Latency Voice Interaction: Traditional speech-to-text followed by text-to-speech pipelines introduce significant delays, making voice interaction feel unnatural.
- Platform Lock-In: Most voice assistants are locked to specific ecosystems (Apple, Google, Amazon) and cannot operate across different operating systems.
- No Visual Awareness: Existing assistants have no ability to see or understand what is currently displayed on the user's screen.

LEO addresses all of these limitations by providing a locally-executed, cross-platform AI assistant with direct system access, persistent memory, autonomous task planning, real-time voice interaction, and visual intelligence.

Chapter 5: Objectives of the Project

Objective 1: Real-Time Voice-Controlled System Interaction

Design and implement a bidirectional real-time voice communication system using the Gemini 2.5 Flash Live API with native audio processing. The system should support continuous conversation with ultra-low latency, enabling users to control their computer entirely through natural voice commands without any manual input.

Objective 2: Modular Action Framework for Comprehensive Computer Control

Develop a modular architecture of specialized action modules that provide complete control over the user's computing environment, including browser automation, file management, application control, system settings, code generation, messaging, and multimedia management.

Objective 3: Autonomous Agentic Task Planning and Execution

Implement an intelligent agentic subsystem capable of decomposing complex, multi-step user goals into sequences of tool calls, executing them autonomously with error recovery mechanisms including retry, skip, replan, and abort strategies.

Objective 4: Persistent Memory and Contextual Awareness

Build a persistent memory system that maintains user context across sessions, storing identity information, preferences, active projects, relationships, and personal notes, enabling LEO to provide personalized and context-aware assistance over time.

Chapter 6: System Architecture

LEO follows a layered architecture pattern with five distinct layers, each responsible for a specific aspect of the system's functionality:

6.1 User Interface Layer

The top layer consists of a custom Tkinter-based HUD (Heads-Up Display) featuring animated rotating rings, pulse effects, audio waveform bars, and a real-time status indicator. Users interact via microphone input or a text command bar.

6.2 Core Engine Layer

The central engine manages the Gemini 2.5 Flash Live API connection, handling bidirectional audio streaming (16kHz PCM input, 24kHz PCM output), tool call dispatching, and session management with automatic reconnection.

6.3 Agentic Layer

Complex multi-step tasks are handled by the agentic subsystem consisting of a Planner (decomposes goals into steps), Executor (runs each step), Error Handler (analyzes failures), and Task Queue (priority-based scheduling).

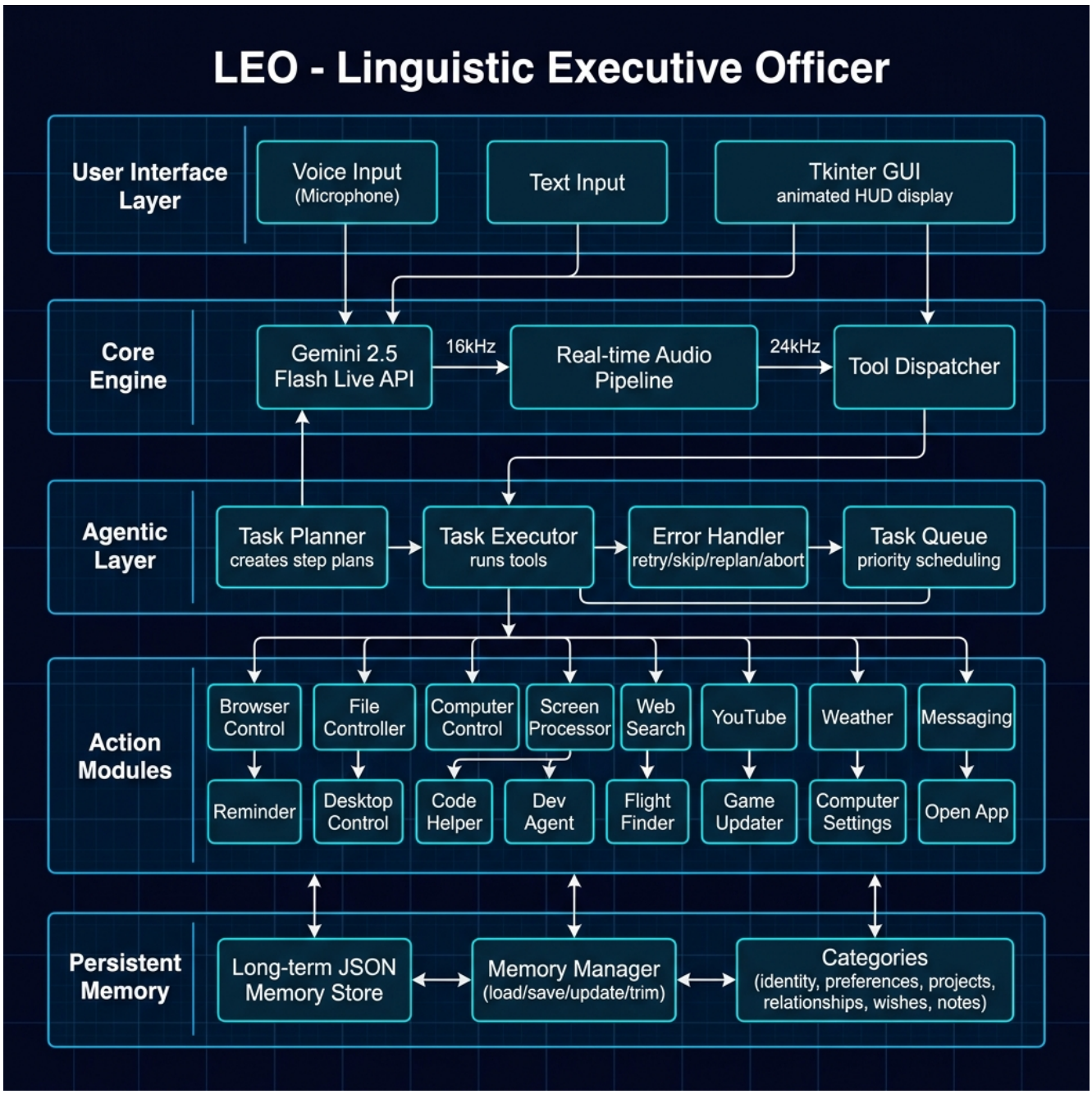
6.4 Action Modules Layer

Sixteen specialized modules handle specific domains: browser_control, file_controller, computer_control, screen_processor, web_search, youtube_video, weather_report, send_message, reminder, desktop_control, code_helper, dev_agent, flight_finder, game_updater, computer_settings, and open_app.

6.5 Persistent Memory Layer

The memory subsystem provides structured long-term storage in JSON format with six categories: identity, preferences, projects, relationships, wishes, and notes. It features automatic trimming to stay within context limits.

System Architecture Diagram


LEO - Linguistic Executive Officer
User Interface Layer
Voice Input (Microphone)
Text Input
Tkinter GUI animated HUD display
Core Engine
Gemini 2.5 Flash Live API
16kHz
Real-time Audio Pipeline
24kHz
Tool Dispatcher
Agentic Layer
Task Planner creates step plans
Task Executor runs tools
Error Handler retry/skip/replan/abort
Task Queue priority scheduling
Action Modules
Browser Control
File Controller
Computer Control
Screen Processor
Web Search
YouTube
Weather
Messaging
Reminder
Desktop Control
Code Helper
Dev Agent
Flight Finder
Game Updater
Computer Settings
Open App
Persistent Memory
Long-term JSON Memory Store
Memory Manager (load/save/update/trim)
Categories (identity, preferences, projects, relationships, wishes, notes)

Chapter 7: Proposed Methodology

7.1 Voice Processing Pipeline

LEO uses a streaming audio architecture where microphone input is captured at 16kHz using the sounddevice library, packetized into 1024-sample chunks, and sent to the Gemini Live API via WebSocket. Responses arrive as 24kHz audio chunks that are played back through a RawOutputStream. The system implements full-duplex communication with speaking detection to mute the microphone during LEO's responses.

7.2 Tool Dispatch Mechanism

When Gemini identifies a user intent requiring action, it returns a tool_call response containing one or more function calls. LEO's dispatcher maps each function name to the corresponding action module and executes it in a thread executor to avoid blocking the async event loop. Results are sent back to Gemini as FunctionResponse objects.

7.3 Agentic Planning Approach

For complex multi-step tasks, LEO employs a Plan-Execute-Recover cycle: (1) The Planner uses Gemini to decompose the goal into max 5 steps with specific tool parameters, (2) The Executor runs each step sequentially with context injection, (3) On failure, the Error Handler classifies the error and decides to retry, skip, replan, or abort, (4) Up to 2 replan attempts are allowed before final failure.

7.4 Memory Management Strategy

User facts are stored via the save_memory tool call, which Gemini invokes silently during conversation. Each memory entry includes a category, key, value, and timestamp. The memory manager enforces a 2200-character limit by trimming the oldest entries first. Memory context is injected into the system prompt at each session start.

Chapter 8: Tools and Technologies Used

Programming Language

Python 3.11/3.12 - Chosen for its rich ecosystem, async/await support, and cross-platform compatibility.

Core AI/ML

- Google Gemini 2.5 Flash Live API - Real-time bidirectional voice + function calling
- google-genai SDK - Primary API client for Live sessions
- google-generativeai SDK - Used for planning and error analysis

UI Framework

- Tkinter - Native Python GUI framework for the animated HUD
- Pillow (PIL) - Image processing for face rendering and scaling

Browser Automation

- Playwright - Cross-browser automation supporting Chrome, Edge, Firefox, Opera, Brave, Vivaldi, Safari

System Control

- PyAutoGUI - Mouse/keyboard automation
- PyGetWindow - Window management
- psutil - Process and system monitoring
- pycaw/comtypes - Windows audio control
- mss - High-performance screen capture
- OpenCV (cv2) - Webcam capture and image processing

Web & Data

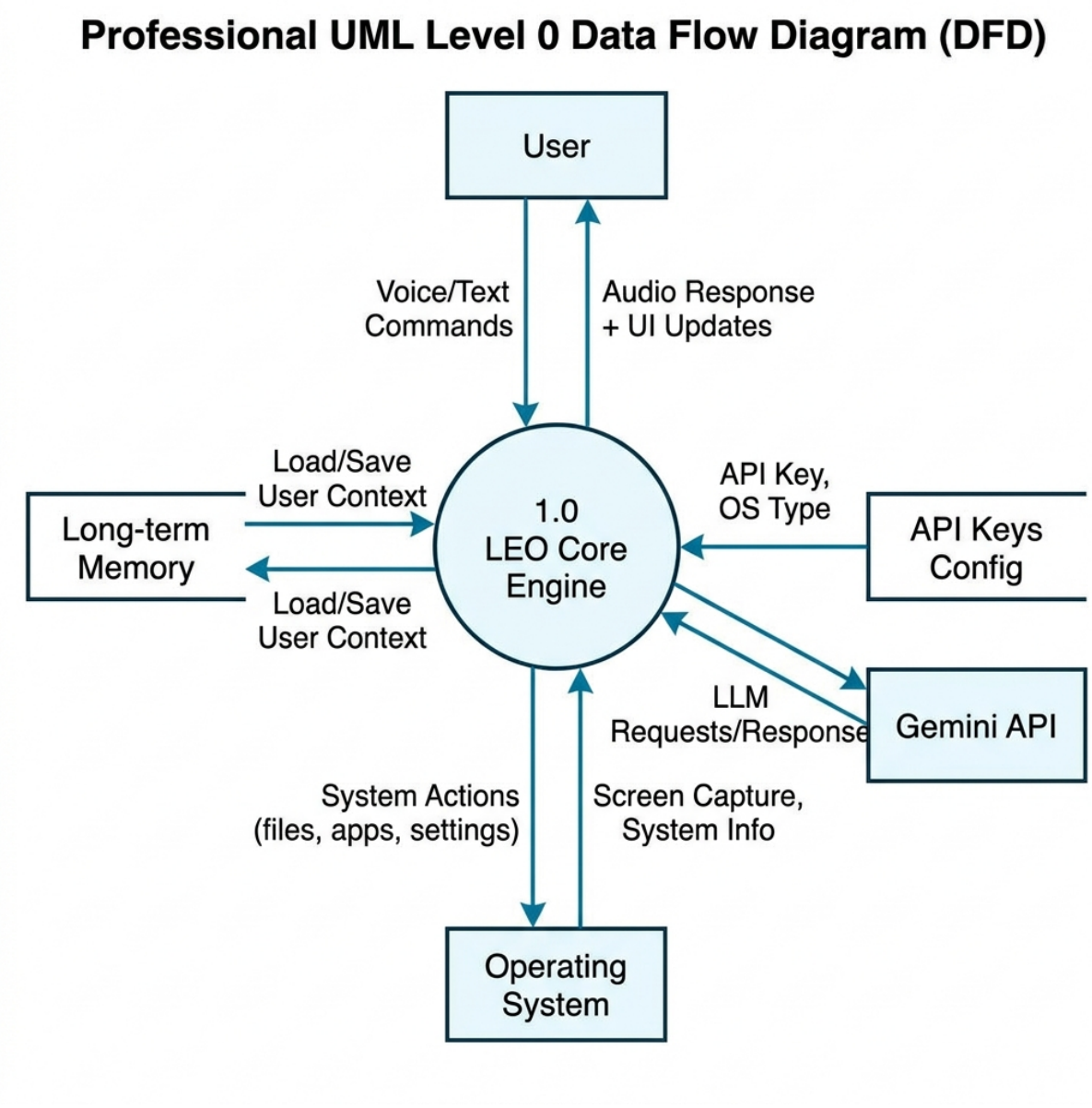
- Requests - HTTP client
- BeautifulSoup4 - HTML parsing
- DuckDuckGo-Search - Web search API
- youtube-transcript-api - YouTube transcript extraction

Development Tools

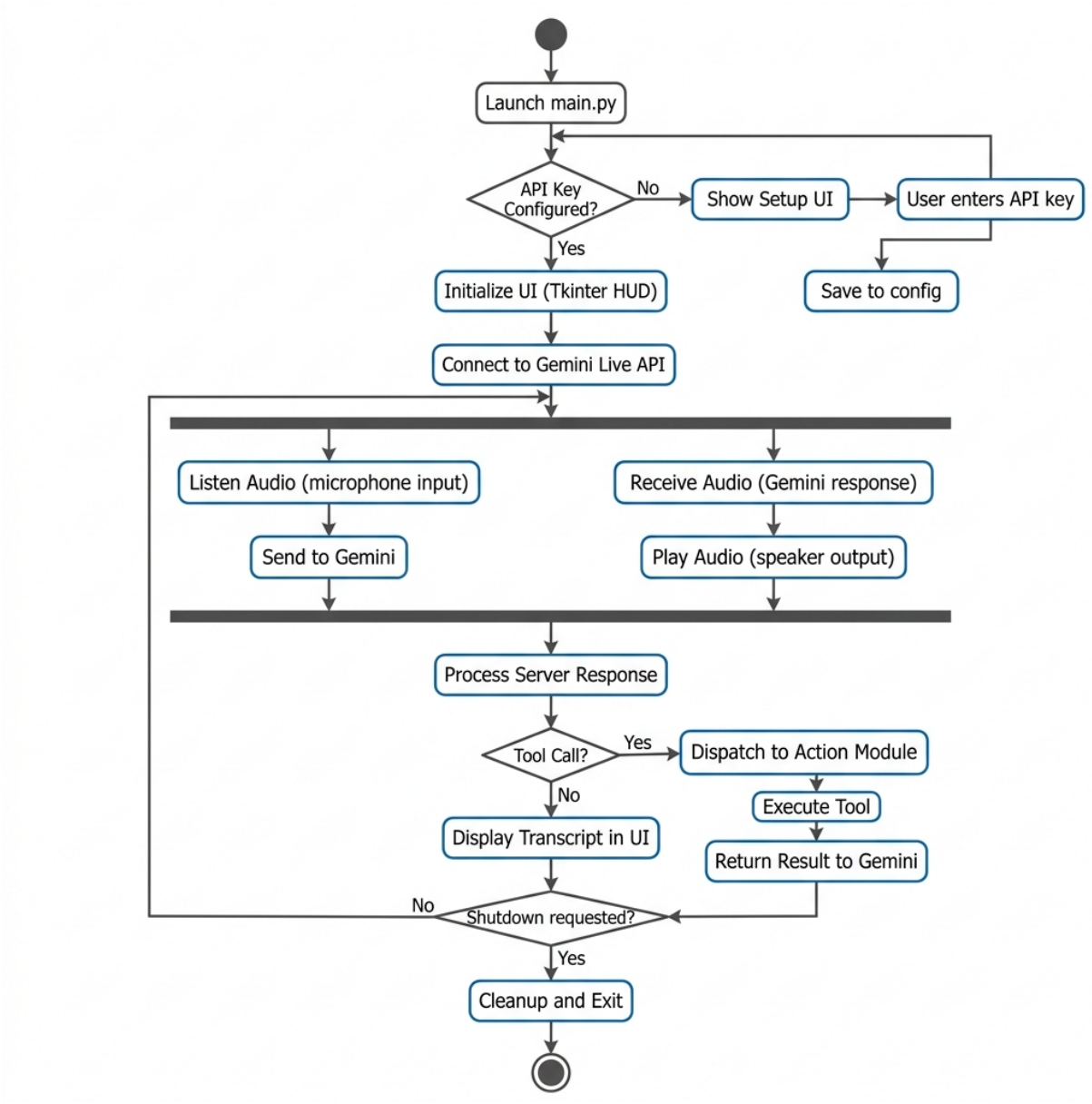
- Git & GitHub CLI - Version control
- VS Code - Primary IDE
- pip - Package management

Chapter 9: Design Diagrams

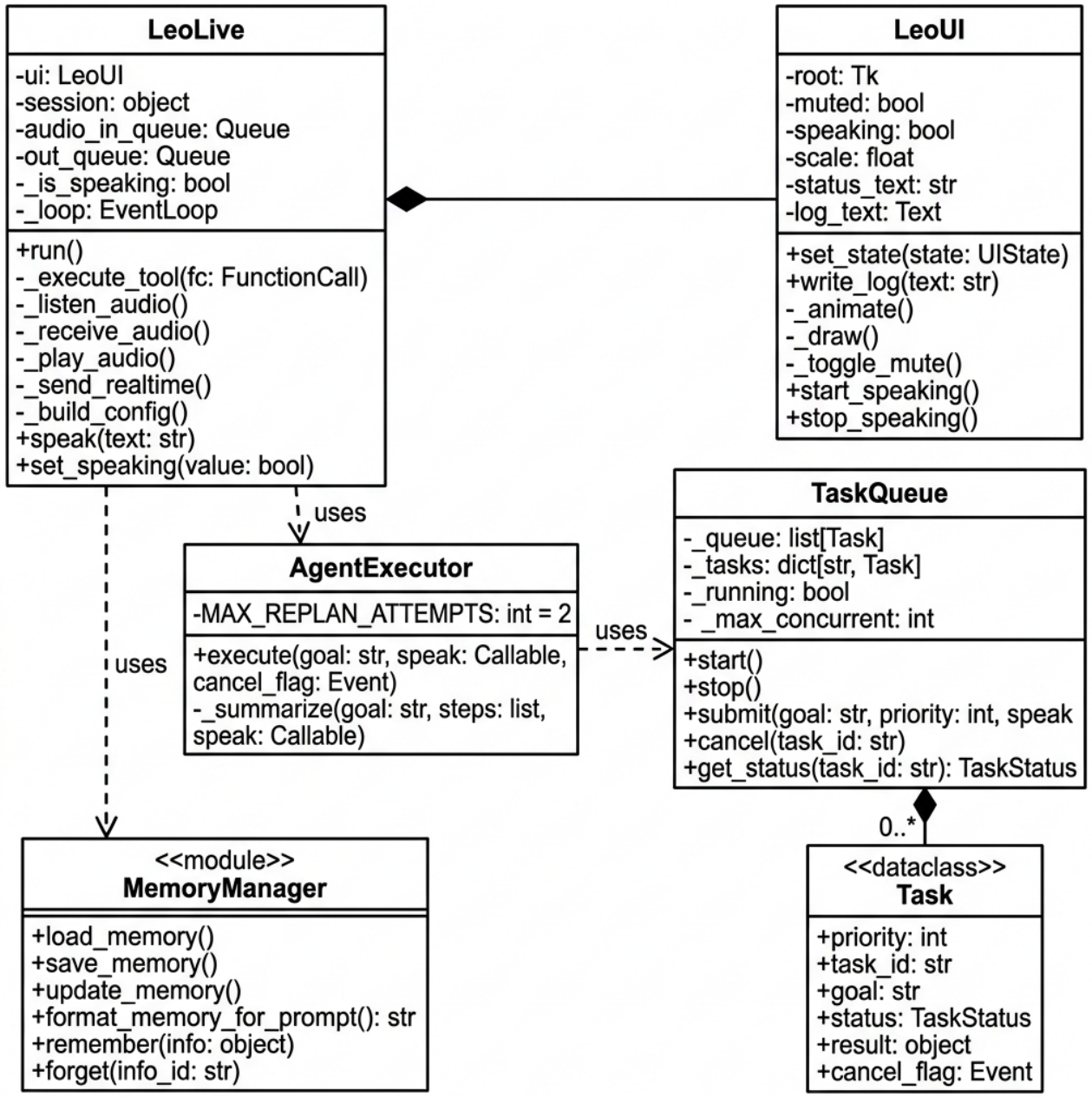
9.1 Data Flow Diagram



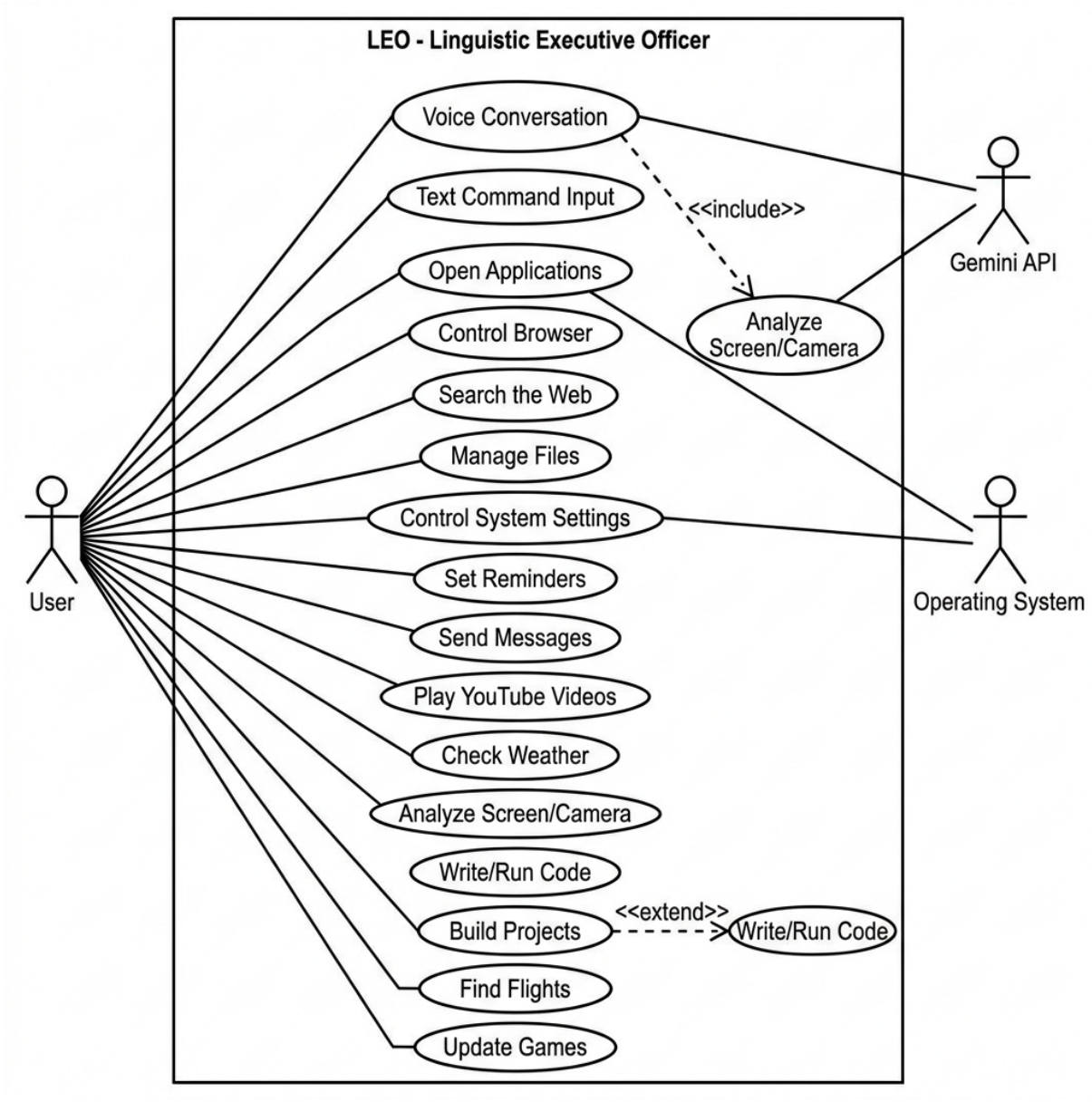
9.2 Activity Diagram



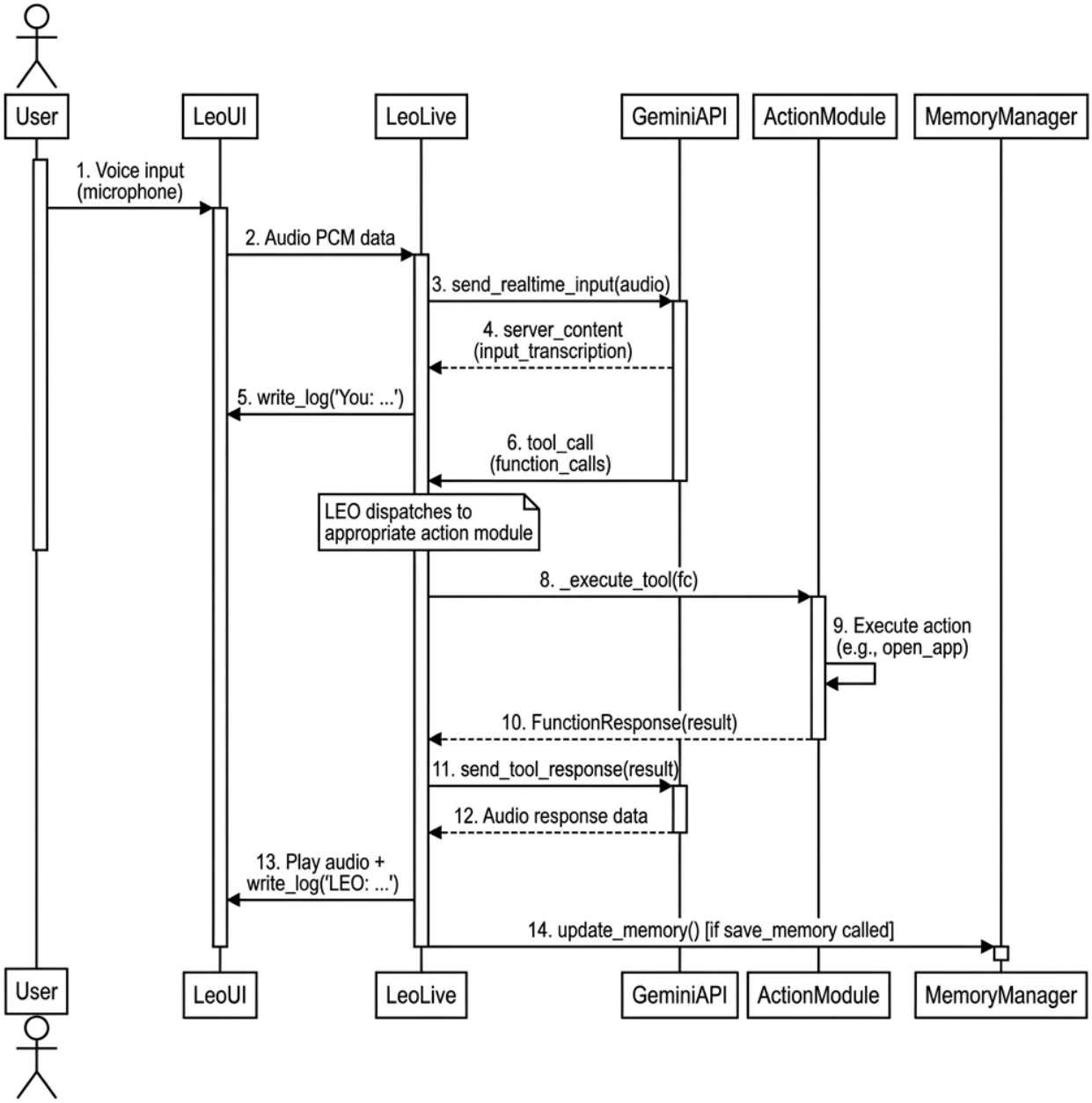
9.3 Class Diagram



9.4 Use Case Diagram



9.5 Sequence Diagram



9.6 Code Snippets

Tool Execution Dispatcher (main.py):

```
async def _execute_tool(self, fc) -> types.FunctionResponse:
    name = fc.name
    args = dict(fc.args or {})
    self.ui.set_state("THINKING")
    if name == "save_memory":
        category = args.get("category", "notes")
        key = args.get("key", "")
        value = args.get("value", "")
        if key and value:
            update_memory({category: {key: {"value": value}}})
        return types.FunctionResponse(
            id=fc.id, name=name,
            response={"result": "ok", "silent": True})
    loop = asyncio.get_event_loop()
    result = "Done."
    if name == "open_app":
        r = await loop.run_in_executor(None,
            lambda: open_app(parameters=args, player=self.ui))
        result = r or f"Opened {args.get('app_name')}."
    # ... dispatch to other action modules ...
```

Memory Manager - Load and Format (memory_manager.py):

```
def load_memory() -> dict:
    if not MEMORY_PATH.exists():
        return _empty_memory()
    with _lock:
        data = json.loads(MEMORY_PATH.read_text(encoding="utf-8"))
        if isinstance(data, dict):
            base = _empty_memory()
            for key in base:
                if key not in data:
                    data[key] = {}
            return data
        return _empty_memory()

def format_memory_for_prompt(memory: dict) -> str:
    lines = []
    identity = memory.get("identity", {})
    for field in ["name","age","city","job","language"]:
        entry = identity.get(field)
        if entry:
            val = entry.get("value") if isinstance(entry,dict) else entry
            if val: lines.append(f"{field.title()}: {val}")
    header = "[WHAT YOU KNOW ABOUT THIS PERSON]\n"
    return header + "\n".join(lines) + "\n"
```

Agentic Planner (agent/planner.py):

```
def create_plan(goal: str, context: str = "") -> dict:
    genai.configure(api_key=_get_api_key())
    model = genai.GenerativeModel(
        model_name="gemini-2.5-flash-lite",
        system_instruction=PLANNER_PROMPT)
    user_input = f"Goal: {goal}"
    response = model.generate_content(user_input)
    text = response.text.strip()
    plan = json.loads(text)
    # Validate and sanitize plan steps
```

```
    for step in plan["steps"]:
        if step.get("tool") in ("generated_code",):
            step["tool"] = "web_search"
            step["parameters"] = {"query": desc[:200]}
    return plan
```

Chapter 10: Progress of Work

The LEO project has been developed iteratively through multiple development cycles:

Phase 1: Foundation (Mark I - Mark X)

Initial prototyping of voice interaction using speech recognition libraries. Basic command parsing and application launching. Simple rule-based intent detection.

Phase 2: Intelligence Integration (Mark XI - Mark XXV)

Migration to Google Gemini API for natural language understanding. Implementation of tool calling mechanism. Development of core action modules (open_app, web_search, weather, reminders).

Phase 3: Agentic Capabilities (Mark XXVI - Mark XXXV)

Implementation of the agentic planning subsystem with Planner, Executor, and Error Handler. Addition of browser automation via Playwright. Development of computer control and screen processing modules.

Phase 4: LEO Release (Mark XXXVII)

Migration to Gemini 2.5 Flash Live API with native audio for real-time voice. Complete UI overhaul with animated HUD. Implementation of persistent memory system. Addition of dev_agent, code_helper, game_updater, and flight_finder modules. Cross-platform support for Windows, macOS, and Linux. Full rebranding from Jarvis to LEO. Published to GitHub as VARUN-NAYAKA/LEO.

Chapter 11: Modules Planned and Completed

Completed Modules

Module	Description	Status
main.py	Core engine - Gemini Live API connection, audio pipeline, tool di	Completed
ui.py	Animated Tkinter HUD with status indicators and waveform visualiz	Completed
browser_control.py	Multi-browser automation via Playwright (Chrome, Edge, Firefox, O	Completed
file_controller.py	File/folder CRUD, search, disk usage, desktop organization	Completed
computer_control.py	Direct mouse, keyboard, hotkey, and screen element automation	Completed
computer_settings.py	Volume, brightness, WiFi, dark mode, window management, shortcuts	Completed
screen_processor.py	Real-time screen capture and webcam analysis via Gemini Vision	Completed
web_search.py	DuckDuckGo-based web search with comparison mode	Completed
youtube_video.py	Video playback, summarization, trending, and info retrieval	Completed
send_message.py	WhatsApp/Telegram messaging via UI automation	Completed
reminder.py	Task Scheduler-based timed reminders	Completed
desktop_control.py	Wallpaper, organize, clean, and desktop statistics	Completed
code_helper.py	Write, edit, explain, run, and build code files	Completed
dev_agent.py	Full project scaffolding with dependency installation	Completed
flight_finder.py	Google Flights search and comparison	Completed
game_updater.py	Steam and Epic Games update/install management	Completed
agent/planner.py	LLM-based goal decomposition into tool-call sequences	Completed
agent/executor.py	Sequential step execution with context injection	Completed
agent/error_handler.py	Error classification and recovery (retry/skip/replan/abort)	Completed
agent/task_queue.py	Priority-based concurrent task scheduling	Completed
memory/memory_manager.py	Persistent JSON memory with CRUD and auto-trimming	Completed

Future Enhancements (Planned)

- Email integration (Gmail, Outlook) for reading and composing emails
- Calendar synchronization with Google Calendar and Outlook
- Smart home device control via IoT protocols
- Multi-language TTS with voice cloning capabilities
- Plugin system for community-contributed action modules

Chapter 12: References

[1] Google, 'Gemini API Documentation - Live API,' 2025. https://ai.google.dev/gemini-api/docs/live

[2] Google, 'Gemini 2.5 Flash Model Card,' 2025. https://ai.google.dev/gemini-api/docs/models

[3] Microsoft, 'Playwright Python Documentation,' 2024. https://playwright.dev/python/

[4] Python Software Foundation, 'Tkinter Documentation,' 2024. https://docs.python.org/3/library/tkinter.html

[5] A. Sweigart, 'PyAutoGUI Documentation,' 2024. https://pyautogui.readthedocs.io/

[6] sounddevice Contributors, 'python-sounddevice Documentation,' 2024. https://python-sounddevice.readthedocs.io/

[7] OpenCV Team, 'OpenCV-Python Documentation,' 2024. https://docs.opencv.org/

[8] oleksis, 'mss - Multiple Screen Shots,' 2024. https://python-mss.readthedocs.io/

[9] S. Russell and P. Norvig, 'Artificial Intelligence: A Modern Approach,' 4th Ed., Pearson, 2021.

[10] A. Vaswani et al., 'Attention Is All You Need,' NeurIPS, 2017.